Associative Context Memory: コンテキストウィンドウ依存を超える LLM コーディングエージェントのための永続的体験ベースメモリ

Hiroshi Yamato
Keio University / Signal compose Inc.
yamato@signalcompose.com

***Abstract*—大規模言語モデル（LLM）ベースのコーディングエージェントは、根本的な制約を抱えている。その記憶はコンテキストウィンドウに完全に束縛されており、トークン数の増加に伴い品質が劣化する——Context Rot として知られる現象である。コンテキスト圧縮、セッション分割、検索拡張生成（RAG）といった既存の緩和策は症状に対処するものの、コンテキストウィンドウサイズへの根本的な依存を解消するものではない。本研究では、Associative Context Memory (ACM) を提案する。ACM は LLM コーディングエージェントのための外部永続メモリアーキテクチャであり、暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉し、コンテキストウィンドウ容量に依存することなくセッションをまたいだ学習を可能にする。本研究の主要な貢献は、コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系とその強度序列である。ACM を MCP（Model Context Protocol）サーバーとして設計し、既存の LLM コーディングエージェントエコシステムとの互換性を確保する。本論文は ACM をポジションペーパーおよびシステム提案として提示するものであり、Phase 0 実現可能性プローブを含む。**

I. はじめに

Claude Code や OpenAI Codex に代表される LLM ベースのコーディングエージェントは、ソフトウェア開発タスクにおいて顕著な能力を示している。しかし、これらは根本的なアーキテクチャ上の制約を共有している。すなわち、すべてのコンテキスト——会話履歴、ツール出力、システムプロンプト、検索されたドキュメント——が有限のコンテキストウィンドウに収まる必要がある。セッション内でトークン数が増加するにつれ、モデルの性能は測定可能な形で劣化する。近年の実証研究はこの現象を **Context Rot** [1] として体系的に特徴づけ、トークン全体に注意が分散し、コンテキストが蓄積されるにつれてタスク成功確率が低下することを示している。

この問題に対する現在のアプローチは 2 つのカテゴリに分類される。

圧縮・分割アプローチ（auto-compact、Plan Mode、手動セッションリセット）は、Context Rot をセッション管理の問題として扱う。コンテキストサイズの削減により劣化を緩和するが、セッションをまたいで知識を引き継ぐ機構を持たない。新しいセッションはすべてゼロから始まる——我々はこれを「**健忘エージェント（amnesiac agent）問題**」と呼ぶ。

検索拡張アプローチ（Serena [2]、RAG ベースのコンテキスト拡張）は、エージェントの関連情報へのアクセスを改善するが、何が成功し何が失敗したかという経験的知識の捕捉は行わない。

いずれのカテゴリも、より深い問いに答えていない。**エージェントは自身のインタラクション履歴から、コンテキストウィンドウとセッションをまたいで持続する形で学習できるか？**

人間の認知は示唆に富む対照を提供する。経験豊富な開発者は過去のデバッグセッションの逐語的な記憶を保持しない。代わりに連想パターンを蓄積する。この知識は疎で、圧縮されており、連想検索の手がかりによってインデックスされている。

本研究では、LLM コーディングエージェントのための永続的な外部メモリ層である **Associative Context Memory (ACM)** を提案する。ACM は以下を実現する。

1) 暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉する
2) 連想キーでインデックスされた構造化・検索可能な形式で体験を保存する
3) 過度なコンテキスト消費なしに、関連する過去の体験を新しいセッションに注入する
4) 持続的な性能のためのコンテキストウィンドウサイズへの依存を低減する

本研究の中心的な新規性は、コーディングエージェントインタラクションにおいて利用可能な**暗黙的フィードバックシグナルの分類体系**である。我々は、インタラプトイベント——Ctrl+C または同等の手段によるエージェント実行の強制停止——を、特にその直後の対話と組み合わせた場合に、ユーザー不満の最も強力なシグナルとして同定する。このシグナルは PTY（擬似端末）レベルで容易に観測可能であるにもかかわらず、先行研究では見過ごされてきた。

II. 背景と関連研究

A. コンテキストウィンドウの制約と *Context Rot*

Transformer アーキテクチャの self-attention 機構はコンテキストウィンドウ内のすべてのトークンに対して動作し、計算量はトークン数 n に対して $O(n^2)$ でスケーリングする。Chroma Research [1] はこれを「Context Rot」として体系的に特徴づけ、Claude Opus 4 や Sonnet 4 を含む最先端モデルであってもコンテキストが充填されるにつれて測定可能な性能劣化を示すことを実証した。

実務者の間では、コンテキストウィンドウの最初の約40%（「Smart zone」）の範囲内で性能が最も信頼できるという経験則が観察されている [3]。

B. コーディングエージェントにおけるコンテキスト管理

Auto-compact: コンテキストが容量に近づいた際のセッション履歴の要約。重大な弱点として、要約がコンテキスト容量の限界付近で動作する劣化したモデルによって実行される。

Plan Mode (Claude Code): 計画と実装を別個のセッションに分離する。効果的だが、手動のワークフロー規律を必要とし、クロスセッションメモリを提供しない。

CLAUDE.md と永続的な指示 [4]: セッション開始時に読み込まれる静的ドキュメント。安定したプロジェクトコンテキストには有用だが、インタラクション履歴に適応しない。

C. エージェントメモリシステム

LLM エージェントのための永続メモリの研究領域は急速に発展している。

MemGPT / Letta [5] は、コンテキストウィンドウを「RAM」、外部ストレージを「ディスク」として扱う階層型メモリアーキテクチャを導入した。最も影響力のある汎用エージェントメモリアーキテクチャであるが、結果に基づく重み付けメモリ選択やコーディング固有のフィードバックシグナルには対応していない。

Mem0 および A-MEM [6] は、セマンティック検索とグラフベースの組織化を備えたメモリ層を提供する。A-MEM [7] は動的なノート構築とリンク生成を通じて柔軟性を向上させている。いずれのシステムもメモリエントリを結果の品質によって区別しない。

A-MAC [8] は、ACM に構造的に最も類似した先行研究である。メモリ受入を 5 つの価値シグナルに分解するが、コーディングエージェントインタラクションからの行動シグナルを受入基準として利用しない。

REMEMBERER [9] は、Q 値を含むインタラクション記録としてエピソディックメモリを実装し、ベース LLM のファインチューニングなしに強化学習によって更新する。

Self-Generated In-Context Examples [10] は、成功したタスク軌跡を in-context examples として保存し、ALFWorld で 73%→89% の性能向上を実証している。ACM はこの方向を拡張し、(1) 失敗体験を含め、(2) シグナル強度分類体系を導入し、(3) コーディングエージェントに特化する。

Letta Code [11] は、MemGPT/Letta アーキテクチャに基づくメモリファーストのコーディングエージェントである。Terminal-Bench で優れた結果を達成しているが、PTY レベルの行動シグナルをメモリ品質指標として活用していない。

How Memory Management Impacts LLM Agents [12] は、LLM エージェントの「experience-following property」——エージェントが誤ったものを含むメモリからのパターンを複製する傾向——を実証的に示している。この知見は ACM における成功・失敗エントリの区別を直接的に動機づける。

Reflexion [13] は、タスク試行の失敗後に言語的な自己フィードバックを生成し、エピソディックメモリとして保存する。重要な違いとして、Reflexion はエージェント自身の自己省察によってフィードバックを生成するのに対し、ACM はユーザーからの暗黙的行動シグナルを捕捉する。また、Reflexion はメモリを検索可能な外部ストアとしてセッション間で永続化しない。[1]

ExpeL [14] は、過去のタスク軌跡から転移可能な知識を抽出する。ACM はリアルタイムの暗黙的フィードバックシグナルに焦点を当てる点で異なる。ExPeL の回顧的軌跡分析にはレイテンシが伴い、知識抽出はタスク完了後に行われるため、セッション中には利用できない。ACM のリアルタイムシグナル捕捉は、将来の拡張においてセッション内の行動調整を支援し得る。

Voyager [15] は、Minecraft 環境における成功したタスク実行から永続的なスキルライブラリを構築する。ACM はこの方向をコーディングエージェントインタラクションに拡張し、暗黙的ユーザーフィードバックを使用して失敗体験も追加する。

すべての先行メモリシステムに共通する制約は、**PTY レベルの行動シグナル**——インタラプトイベント、rewind 操作、修正指示パターン——をファーストクラスのメモリ品質シグナルとして**欠いている**ことである。

D. 人間-*LLM* インタラクションにおける暗黙的フィードバック

ICML 2025 の論文 [16] は、WildChat および LMSYS データセットにおける暗黙的ユーザーフィードバックを分析し、フィードバックの内容が極性のみよりもモデル性能を改善することを発見した。Google での並行研究 [17] は、Cider Chat のテレメトリを分析し、明示的フィードバックがインタラクションのわずか 0.6% でしか発生しないことを発見した。

いずれの研究も PTY レベルのインタラプトイベントをフィードバックシグナルとして検討していない。ACM のシグナル分類体系は、ターミナルベースのコーディングエージェントに特化してこのギャップを埋める。

E. 検索拡張コード生成

Serena [2] は、LSP を使用してコードベースのセマンティックインデックスを構築する。ACM は Serena と直交する。Serena はコードベースのどの情報が利用可能かを改善し、ACM はこのコードベースでの作業に関するどの経験的パターンが利用可能かを改善する。

F. ACM の位置づけ

表 Iに ACM と関連研究の比較を示す。

[1]Reflexion は単一タスク内のリトライ間でエピソディックメモリを永続化するが、異なるタスクやコードベース間で利用可能な汎用的検索可能ストアを提供しない。

TABLE I
ACM と関連エージェントメモリシステムの比較

システム	クロスセッションメモリ	失敗体験	PTY/行動シグナル	コーディング特化
MemGPT / Letta	Yes	No	No	No
Mem0 / A-MEM	Yes	No	No	No
A-MAC	Yes	Partial	No	No
REMEMBERER	Yes	Yes (Q 値)	No	No
Self-Gen ICE	Yes	No	No	No
Reflexion	No*	Yes	No	No
ExpeL	Yes	Yes	No	No
Voyager	Yes	No	No	No
Letta Code	Yes	No	No	Yes
ACM（本研究）	**Yes**	**Yes**	**Yes**	**Yes**

* セッション内のみ。脚注参照。

III. ACM フレームワーク

A. アーキテクチャ概要

ACM は、既存の LLM コーディングエージェントインフラストラクチャをラップする MCP（Model Context Protocol）サーバーとして設計される。この設計選択により、Claude Code、OpenAI Codex CLI、および MCP をサポートする任意のエージェントとの互換性が確保される。

Listing 1. ACM MCP Server hook アーキテクチャ

```
[ACM MCP Server] -- Claude Code hooks API

  SessionStart hook
    -> ACMコンテキスト注入

  UserPromptSubmit hook
    -> インタラプト後N=3-5ターン捕捉
    -> 修正指示パターン検知

  PostToolUse hook
    -> 成功したツール完了を記録

  PostToolUseFailure hook
    -> is_interrupt=true: 失敗エントリ生成開始

  Stop hook
    -> 正常完了: 成功候補
    -> 非発火 = 補完的インタラプト検知

  SessionEnd hook
    -> エントリ確定・永続化
```

ACM 層は 3 つの機能を実行する。

1) **セッション開始時の検索**: 現在のタスクに関連するエントリについて体験 DB を照会し、圧縮された表現をセッションコンテキストに注入する
2) **セッション中のシグナル監視**: PTY インターフェースと会話から暗黙的フィードバックシグナルを捕捉する
3) **セッション終了時の体験書き込み**: セッションをスコアリングし、成功または失敗エントリとして保存する

B. 体験エントリ構造

ACM は 2 種類のエントリを管理する。

成功エントリ（「これをせよ」）：

```
{
  "type": "success",
  "trigger": "Authentication bug in auth.py",
  "action": "Used FileSystemWatcher, showed
            diff before applying",
  "outcome": "Tests passed, no rewind",
  "retrieval_keys": ["auth", "JWT",
    "FileSystemWatcher", "diff"],
  "signal_strength": 0.85
}
```

失敗エントリ（「これをするな」）：

```
{
  "type": "failure",
  "trigger": "Authentication bug in auth.py",
  "action": "Attempted bulk rewrite of all
            related files simultaneously",
  "outcome": "Interrupt detected. Post-
    interrupt: 'just change the diff'",
  "retrieval_keys": ["auth", "bulk rewrite"],
  "signal_strength": 0.95,
  "interrupt_context": {
    "turns_captured": 3,
    "dialogue_summary": "User wanted
      incremental diff-based changes"
  }
}
```

C. 検索と注入

各セッション開始時に、ACM はユーザーの初期タスク記述からキーワードを抽出し、体験 DB の `retrieval_keys` に対してベクトル類似度検索を実行し、上位 K 件の関連エントリを検索して圧縮サマリーをセッションコンテキストに注入する。

注入は意図的にコンパクトである。詳細なエントリは外部に保存されパスで参照されるため、コンテキストウィンドウの膨張を回避する。これは「インデックスであってコンテンツではない」原則を実装するものである。

D. 体験のスコアリングと昇格

セッション終了時に、ACM は利用可能なシグナル（第 4 節で詳述）を用いてセッションをスコアリングし、昇格閾値を超えるエントリを体験 DB に書き込む。混合シグナルを持つセッションは、成功エントリと失敗エントリの両方を生成する場合がある。

IV. 暗黙的フィードバックシグナル分類体系

本研究の中心的な貢献は、LLM コーディングエージェントインタラクションにおいて利用可能な暗黙的フィードバックシグナルの体系的分類である。

A. *Level 1:* インタラプト + インタラプト後の対話（最高強度）

シグナル: ユーザーがエージェント実行の強制停止（Ctrl+C、Esc×2）をトリガーし、その後に修正的な対話が続く。

Claude Code の `PostToolUseFailure` hook が `is_interrupt` ブーリアンフィールドを提供し、公式 hooks API を通じてユーザー起動のインタラプトを明示的にフラグする。

最強である理由: インタラプトイベントは最もコストの高いユーザーアクション——進行中のプロセスの能動的な停止——を表す。インタラプト後の対話は、ユーザーがなぜ停止したかを明示的に表現することが多いため、特に価値がある。

B. *Level 2: Rewind* 操作（高強度）

シグナル: ユーザーが rewind 機能を呼び出し、会話を以前の状態に復元する。Rewind はエージェント出力の意図的な取り消しを表す——強い暗黙的拒否である。

インタラプトとの区別: Rewind は事後的（エージェントアクション完了後）であり、インタラプトはリアルタイム（実行中）である。

C. *Level 3:* 修正指示（中強度）

シグナル: ユーザーが会話内で自然言語の修正を提供する。言語的修正はインタラプトや rewind よりもコストが低く、軽微なミスアラインメントを反映している場合がある。ただし、セッション内での繰り返しの修正指示（3 回以上）は、rewind イベントと同様に重み付けされるべきである。

D. *Level 4:* 中断なしの完了（ベースライン / 弱い正のシグナル）

シグナル: タスクがインタラプト、rewind、修正指示なしに完了する。負のシグナルの不在は成功の強い確認ではないが、テスト合格と組み合わせた場合は信頼性の高い正のシグナルである。

E. シグナル強度サマリー

注: 以下の強度スコアは実験設計のための仮説値であり、今後の実証データに基づいて較正される。序列はユーザーアクションのコストを反映する。

TABLE II
シグナル強度分類体系

シグナル	スコア	方向	捕捉方法
インタラプト + 対話	0.90–1.00	負	PTY + N ターン
Rewind	0.75–0.90	負	セッションログ
修正指示 (3+)	0.60–0.80	負	会話テキスト
修正指示 (1)	0.30–0.50	負	会話テキスト
テスト通過 + 中断なし	0.70–0.85	正	終了コード
中断なし（テストなし）	0.40–0.60	正	セッションログ

V. 実験設計

A. *Phase 0:* 実現可能性プローブ

本格的な実験評価に先立ち、ACM の主要な技術的前提が Claude Code 環境内で実現可能であるかを検証するための実現可能性プローブを実施した。

知見 1: PTY インタラプトシグナルは公式 hooks API を通じてアクセス可能である。 Claude Code の `PostToolUseFailure` hook は `is_interrupt` ブーリアンフィールドを提供し、ユーザーが Ctrl+C でツール実行を中断した場合に `true` に設定される。

知見 2: セッションログは体験エントリ生成に十分なデータを提供する。セッションログは `~/.claude/projects/`配下に JSONL ファイルとして保存され、完全な会話履歴、ツール呼び出し、ISO 8601 タイムスタンプを含む。

知見 3: セッション後のトークン使用量は回復可能である。トークン使用量はセッション終了後に `~/.claude.json` に記録される。リアルタイムトークン数は hooks API を通じてアクセスできない。

知見 4: Rewind イベントには専用 hook が存在しない。 ACM は `UserPromptSubmit` イベントにおける修正指示パターンとメッセージ数の減少を通じて間接的に rewind を検知する。

B. リサーチクエスチョン

- RQ1 ACM はコンテキストウィンドウ依存を低減しつつ、タスク性能を維持・向上させるか？
- RQ2 ACM-S、ACM-F、ACM-SF のうち、どのメモリ構成が最大の性能向上をもたらすか？
- RQ3 暗黙的フィードバックシグナルの種類は体験エントリ品質と相関するか？
- RQ4 ACM は人為的に制約されたコンテキストウィンドウサイズ下で持続的な性能を可能にするか？
- RQ5 ACM は Serena と補完的か？組み合わせは加算的な効果をもたらすか？

TABLE III
実験条件

条件	説明
Control	ベース LLM エージェント
Baseline-compact	エージェント + auto-compact
Baseline-Serena	エージェント + Serena
ACM-S	ACM（成功エントリのみ）
ACM-F	ACM（失敗エントリのみ）
ACM-SF	ACM（成功 + 失敗）
ACM-SF + Serena	統合アプローチ

C. 実験条件

モデル能力をアーキテクチャ効果から分離するため、固定のオープンウェイトモデルをベースエージェントとして使用する。

D. コンテキストウィンドウ制約条件（*RQ4* 用）

TABLE IV
コンテキストウィンドウ制約

制約	トークン予算
Full	128k（モデルデフォルト）
Half	64k
Smart zone	50k（∼40%）

E. タスクスイート

SWE-bench [18] はシングルセッションタスクであり、ACM のクロスセッションメモリが効果を発揮する条件を含まないため、直接使用しない。本タスクスイートは Context Rot とクロスセッション改善を示すように設計されている。

タスク A — マルチファイルバグ修正: 5〜10 ファイルにわたる変更を必要とする埋め込みバグを修正する。テストスイートの合格で評価。

タスク B — 仕様書からの機能追加: 自然言語の仕様書に記述された機能を実装する。機能テスト + 人間レビューで評価。

タスク C — 設計原則に基づくリファクタリング: 指定された設計原則をコードベース全体に一貫して適用する。自動リンティング + 人間レビューで評価。

各タスクは条件ごとに 5 回の反復セッションで実行される。クロスセッション改善率が主要な指標である。

F. 評価指標

TABLE V
評価指標

指標	測定方法	RQ
タスク完了率	テスト合格 / 人間評価	RQ1
インタラプト回数	PTY SIGINT イベント	RQ1,3
Rewind 回数	セッションイベントログ	RQ1,3
修正指示回数	会話テキスト分析	RQ1,3
コンテキスト効率	トークン / 複雑度	RQ4
クロスセッション改善	Δ 完了率 1→5	RQ1,2
シグナル-品質相関	ピアソン r	RQ3

VI. 議論

A. 健忘エージェント問題

現在の LLM コーディングエージェントは、設計上、健忘的である。ACM は経験的知識をセッションをまたいで持続し蓄積する形式で外部化することにより、この問題に直接対処する。

ACM を備えたエージェントはプロジェクト固有の行動パターンを発達させることができる——コードベースに何が含まれているかだけでなく、この特定のユーザーがこの特定のコードベースで問題にどのようにアプローチすることを好むかを知ることができる。

B. ファーストクラスシグナルとしてのインタラプト

インタラプトイベントは、エージェントシステム研究においてフィードバック機構として十分に活用されていない。コーディングエージェントにおいて、インタラプトは一般的であり意味的に豊かである。誤ったアプローチを追求するエージェントは停止され、ユーザーの後続の説明は、将来そのアプローチを回避するために必要な負の訓練シグナルそのものである。

インタラプト後 N=3〜5 ターンの捕捉は、この対話がユーザーが提供する最も明示的なフィードバックであることが多いため、特に価値がある。

C. 強化学習との関係

ACM のシグナル分類体系は強化学習における報酬シグナルとの構造的類似性を持つが、ACM は基盤モデルに対する重み更新を行わない——コンテキストレベルで動作し、関連する過去の体験をソフトガイダンスとして注入する。この区別により、ACM はファインチューニングインフラストラクチャを必要とせず任意の LLM でデプロイ可能となる。

D. 制約

コールドスタート: ACM は新しいタスクタイプにおける最初のセッションでは効果を発揮しない。性能向上には蓄積された体験が必要である。

体験の陳腐化: コードベースの進化により、過去の体験が誤解を招く可能性がある。

インタラプトの曖昧性: すべてのインタラプトがエージェントのアプローチへの不満を示すわけではない——外部的な中断を反映するものもある。

適用範囲: 現在の設計はコーディングエージェントに特化している。他のドメインへの一般化にはシグナル分類体系の適応が必要である。

Rewind 検知: Claude Code は rewind 操作のための専用 hook イベントを提供していない。ACM は

`UserPromptSubmit` イベントにおける修正指示パターンまたはメッセージ数の減少を通じて間接的に rewind を検知する。

リアルタイムコンテキスト使用量: アクティブセッション内のトークン数は hooks API を通じてアクセスできない。ACM はセッション後トークン数を使用し、`PreCompact` hook をニアリミット条件のプロキシとして扱う。

VII. 結論

本研究では、暗黙的なユーザーフィードバックから成功・失敗体験を捕捉する LLM コーディングエージェントのための永続的外部メモリアーキテクチャである Associative Context Memory (ACM) を提示した。ACM の主要な貢献は以下の通りである。

1) コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系。インタラプト + インタラプト後の対話を利用可能な最強のシグナルとして同定した
2) 成功（「これをせよ」）と失敗（「これをするな」）を区別する体験エントリ構造。エージェントが正のパターンと回避行動の両方を発達させることを可能にする
3) モデル修正なしに既存の LLM コーディングエージェントエコシステムと統合する MCP 互換のアーキテクチャ設計
4) クロスセッション学習とコンテキストウィンドウ依存低減を評価するための実験設計

本研究の中心的主張——アーキテクチャ的メモリがエージェントのコンテキストウィンドウ容量への依存を低減できるということ——は、エンジニアリング実務を超えた含意を持つ。検証されれば、「健忘エージェント問題」がコンテキストウィンドウのスケーリングなしに対処可能であることを示唆し、持続的なエージェント性能への補完的な道筋を提供する。

論文草稿および関連ドキュメントはhttps://github.com/signalcompose/okitegami_paperで公開している。

著者貢献および *AI* 利用開示

知的貢献と著者責任。本論文におけるすべてのリサーチクエスチョン、中核的アイデア、アーキテクチャ概念、および知的貢献は Hiroshi Yamato に帰属する。著者は本論文に記載された内容に対して全責任を負う。

研究・執筆過程における AI ツールの使用。本論文は、人間主導の AI 協働ワークフローを通じて制作された。

- **Claude.ai (Anthropic)** は *Strategist* として機能した：セッション間の会話コンテキストの維持、文献検索と統合の支援、査読シミュレーション、およびテキストの起草・改訂の補助を担った。
- **Claude Code (Anthropic)** は *Implementer* として機能した：ファイル操作の実行、論文メタデータ取得のためのウェブ検索、草稿ドキュメントの更新、およびバージョン管理を担った。

AI ツールが行わなかった事項：リサーチクエスチョンの着想、ACM アーキテクチャの提案、インタラプトをシグナルとして活用する着想の発見、論文の範囲や主張に関する意思決定。AI が生成したテキストはすべて、掲載前に著者によるレビュー・承認を経ている。プロセスの全容は、付随リポジトリの `docs/session-log.md` に記録されている。

補遺 — 2026-03-08：本原稿の準備過程において、第 1 節で述べたセッションログ維持の問題が著者自身のワークフローで観察された。本プロジェクトで使用した Strategist/Implementer 協働パターン—claude.ai（Strategist）がセッションコンテキストを保持し、Claude Code（Implementer）がファイル操作を実行する—では、`docs/session-log.md` を最新に保つために手動のブリッジングが必要であった。この手動更新ステップは頻繁に漏れが生じ、セッション再開時にコンテキストのギャップが発生した。

この問題は、Claude Code の Stop フック（`.claude/hooks/update_session_log.py`）を実装することで対処した。このフックは Claude Code が応答ターンを完了するたびに自動発火し、編集されたファイルの記録を `session-log.md` に追記する。フックのデバッグでは、実際の JSONL トランスクリプト構造—`tool_use` ブロックがトップレベルではなく `assistant.message.content[]` 内にネストされている—を考慮する必要があった。この一人称的観察—「記憶喪失エージェント」問題が本論文の執筆ワークフロー自体にも顕在化するという事実—は、第 1 節で提示した ACM の動機をさらに補強するものである。

References

[1] Chroma Research, "Context rot: How increasing input tokens impacts LLM performance," https://research.trychroma.com/context-rot, 2025, technical report.

[2] Oraios AI, "Serena: A coding agent toolkit providing semantic code retrieval and editing via LSP and MCP," https://github.com/oraios/serena, 2025, open-source software.

[3] G. Huntley, "How to build a coding agent," https://ghuntley.com/agent/, 2025, blog post. Video: https://www.youtube.com/live/fOPvAPdqgPo.

[4] Anthropic, "Effective context engineering for AI agents," https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025, engineering blog post.

[5] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[6] P. Chhikara *et al.*, "Mem0: The memory layer for personalized AI," 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[7] W. Xu *et al.*, "A-MEM: Agentic memory for LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[8] G. Zhang, W. Jiang, X. Wang, A. Behr, K. Zhao, J. Friedman, X. Chu, and A. Anoun, "Adaptive memory admission control for LLM agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.04549

[9] Y. Zhang *et al.*, "REMEMBERER: Equipping large language models with persistent memory via long-term interaction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2306.07929

[10] V. Sarukkai *et al.*, "Self-generated in-context examples improve LLM agents for sequential decision-making," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: https://arxiv.org/abs/2505.00234

[11] Letta, "Letta code: Memory-first coding agent," https://www.letta.com/blog/letta-code, 2025, accessed 2026-03-08.

[12] Z. Xiong *et al.*, "How memory management impacts LLM agents: An empirical study of experience-following behavior," 2025. [Online]. Available: https://arxiv.org/abs/2505.16067

[13] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[14] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2308.10144

[15] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.16291

[16] Y. Liu, M. J. Q. Zhang, and E. Choi, "User feedback in human-LLM dialogues: A lens to understand users but noisy as a learning signal," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2507.23158

[17] D. Nam, M. Salawa, and S. Chandra, "Reading between the lines: Scalable user feedback via implicit sentiment in developer prompts," 2025. [Online]. Available: https://arxiv.org/abs/2509.18361

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770